Embedded Systems International

Project Statement of Work (PD3)

Team Identifier: D3

Team Name (optional): Oreos

Team Member Names: Tyler Holtz, Keaton Hy, Jacob Barglof, Ridhima Gupta

Problem Statement

Define and Frame Your Design Challenge (links to IDF webpage)

Problem Statement:
A rover is needed to survey unfamiliar, harsh territories which are too far away to implement a vision system or advanced communication protocols.
POV statement:
A remote rover operator who cannot directly see the test field needs to monitor robot status, interpret sensor data, and issue movement and scan commands through a clear graphical control station because the rover must be navigated safely through hazards and positioned in the destination zone using only transmitted information.

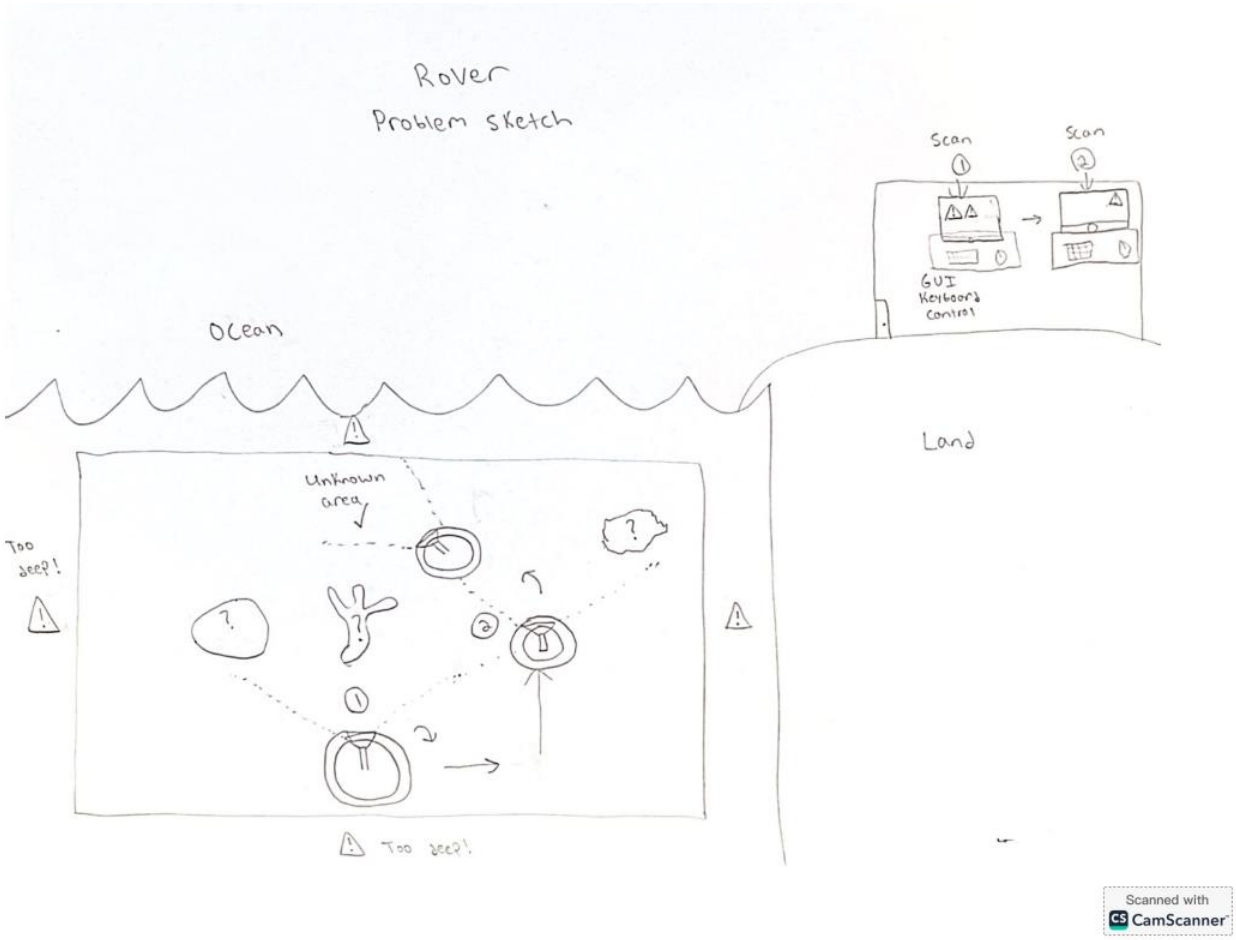
High-level mission goals:
The purpose of the rover system is to allow a human operator to remotely explore and navigate an unknown obstacle field using a GUI-based command center.
The rover will be capable of receiving high-priority movement and scan commands, transmitting sensor and status data back to the GUI, avoiding immediate hazards through onboard safety responses, and being positioned accurately inside the destination zone.
The system will help the operator build an approximate understanding of the environment through scan results, hazard feedback, and field mapping shown in the GUI. These goals align with the project requirement that the base station process transmitted information well enough to navigate the test field and control the robot remotely

Short description of the problem:

The problem is to design a rover and command-station system that can safely traverse an unknown test field even though the operators cannot see the physical course during the demonstration. The rover must provide enough sensor and status data for the base station to understand its surroundings, while also accepting remote commands for motion and scanning. Because the robot can encounter obstacles, holes, and boundaries, the embedded system must enforce safety priorities so that bump and cliff events override normal behavior, and base-station commands take priority over local automation. Our proposed solution is a TM4C-based CyBot rover paired with a Python graphical user interface running on the base station computer. Communication will occur through PuTTY/UART, with sensor data and robot state streamed to the GUI through a serial pipeline. The GUI will allow keyboard-based driving, scan requests, speed tuning, sensor display, and approximate map visualization, while the robot firmware handles motor control, sensor polling, scan execution, and immediate emergency responses. This directly matches the project's requirement for a usable UI, remote control of movement and sensor sweeps, command priority enforcement, and final positioning in the destination zone

In addition to writing a paragraph about the problem, you are to draw a **problem sketch: a one-page sketch illustrating your solution with a user context (big picture view)**. This should show the scope of

your work in relation to one or more user needs. Refer to the sample project sketches.



Scanned with CamScanner

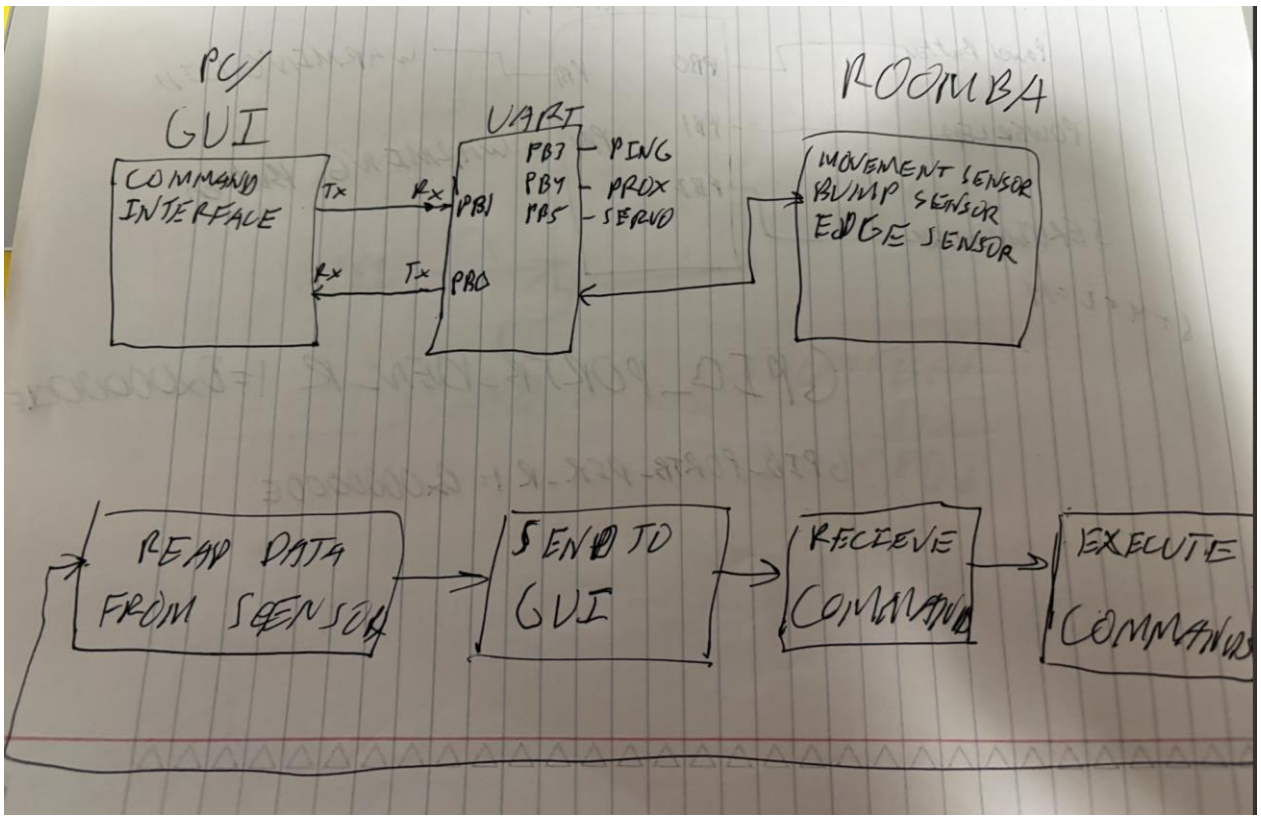
Design Approach

Our design uses a TM4C-based CyBot rover controlled from a Python GUI base station through serial communication. The rover is remotely driven, but it still includes onboard safety responses so that bump and cliff events override all other actions. This matches the project requirement that the base station control the robot while the robot still handles immediate hazards locally.

The system satisfies the required functionality by allowing the operator to move the rover, request scans, monitor bump and cliff sensors, detect obstacles, avoid hazards, stay within boundaries, and position the rover in the destination zone. The GUI serves as the user interface and command center, while the rover returns enough sensor data for navigation in an unseen field.

The required platform components are used as follows: iRobot sensors for bump and cliff detection, interrupts for fast event handling, ADC and input capture for distance sensing, PWM for motor control, and UART for communication between the rover and GUI.

The test field is treated as a hazardous remote-operating environment. Tall and short objects represent obstacles, holes represent dangerous drop-offs, boundaries mark restricted areas, and the pillar-marked zone is the final destination. The design aims to avoid major penalties by prioritizing emergency stops and giving the operator clear feedback through the GUI. It also targets bonus points through the graphical interface and added mapping/control features.



Functionality

Basic Functionality	Mapping to AV Application
Cybot Communication	The rover will maintain two-way communication with the base station so that the GUI can send drive, stop, scan, and mode commands while the robot returns bump status, cliff status, sweep data, and movement/state messages. This matches the required communication and UI-monitoring functions.
Cybot Movement	The rover will move in response to keyboard-driven GUI commands such as forward, backward, left, right, stop, and controlled turning. The operator will also be able to adjust speed and steering sensitivity for finer control in narrow areas.
Object Detection	The rover will detect obstacles using its scan system and onboard sensors, then send that information to the GUI so the operator can

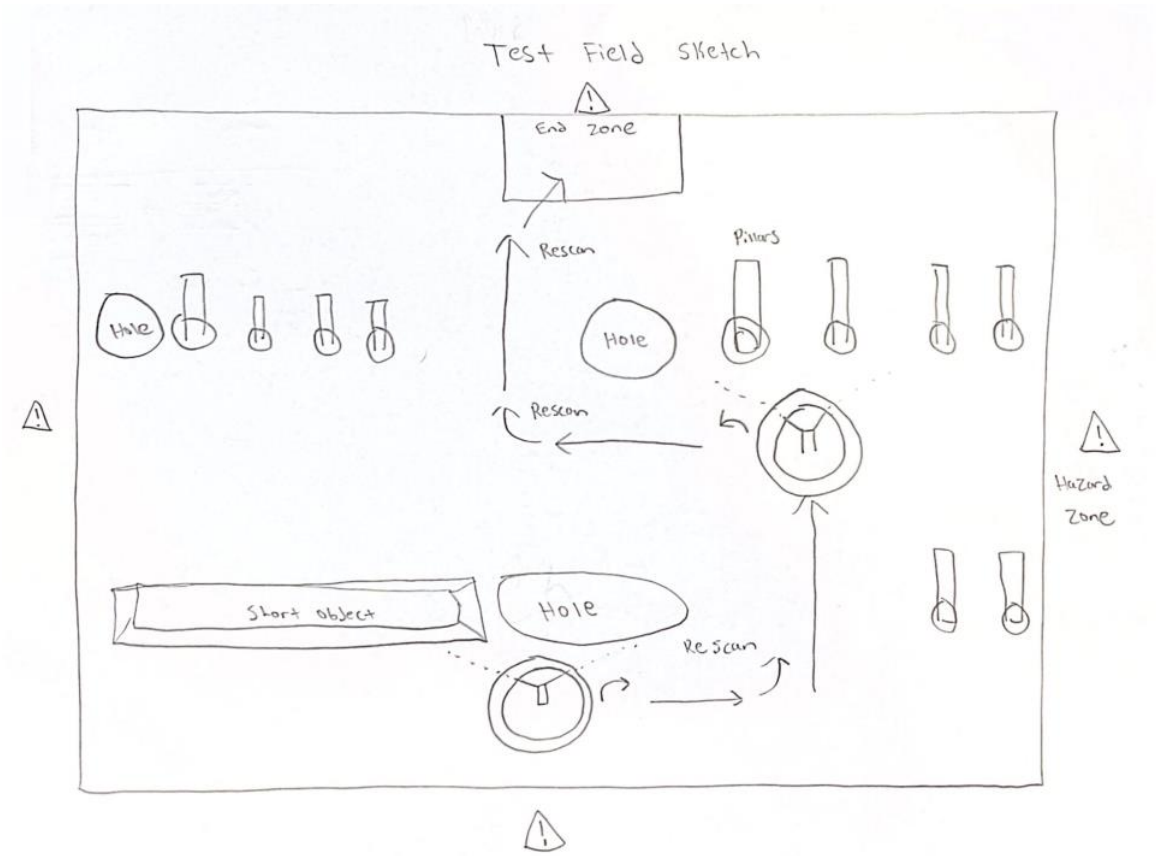
	understand where tall or short objects may be located. This supports safe human-guided navigation through an unseen field. The example Mars rover scenario uses object detection for the same purpose.
Object Avoidance	The main avoidance logic is human-guided through the GUI map and scan results, but the rover will still perform immediate local safety actions when it detects a bump or cliff event. This follows the course requirement that the robot have some onboard decision-making and automatic sensor response.
Boundary Adherence	The rover will detect out-of-bounds markings and cliff-like hazards using its sensors and warn the operator through the GUI. If necessary, the robot will halt or back away automatically to prevent leaving the course or falling into a hole. Boundaries and holes are required test-field elements.
Arrival at Destination	The rover will be guided into the destination zone by the operator using scan data, movement controls, and environment feedback from the GUI. Once the rover is properly positioned inside the zone, it will send a signal confirming arrival, which is the completion requirement for Part III.
User Interface	The base station will use a Python GUI rather than a text-only terminal. The interface will show live robot status, bump/cliff indicators, scan results, drive mode, speed settings, and an approximate field map. This goes beyond the minimum requirement and likely supports GUI bonus credit.
Base Station Control	The operator will control the robot from the GUI using the keyboard and buttons for movement, stopping, sweeping, and mode changes. Base-station commands will be treated as high priority unless superseded by emergency sensor events, exactly as required.
Other Application Specific Functionality (may be novel features for bonus points) The GUI will build an approximate live field map from sweep results so the operator can visually reason about obstacles and the best route to the destination. The operator will be able to tune drive speed and turning sensitivity to improve precise placement in the destination zone. The system will provide a “command center” style display that aggregates sensor state, scan output, and rover status in one place	

Mapping to Platform

Basic Platform Components	Usage in AV Application
Open Interface and iRobot Sensors	Used for bump sensors, cliff sensors, wheel movement, and basic robot driving. These sensors provide the highest-priority safety input and are essential for stopping the robot from hitting hazards or falling into holes.
Interrupts	Used for fast handling of urgent robot events and/or communication timing so that emergency conditions and incoming commands can be serviced quickly rather than waiting in a slow polling loop.
ADC	Used for analog sensor readings such as IR distance sensing, if included in the scan subsystem, so that the rover can estimate object presence and range and transmit that information to the GUI. The project requirements explicitly list ADC as a required platform capability to map.
Input Capture	Used for timing-based distance measurements from sonar/Ping during scans, allowing the robot to determine object distance and send usable sweep data back to the base station.
PWM	Used for motor control and speed regulation so that the operator can tune movement speed and sensitivity from the GUI, enabling both fast traversal and precise destination positioning.
UART/WiFi	Used as the primary communication link between the TM4C/CyBot and the base station. Commands from the GUI will be sent over serial through PuTTY, and the robot will return sensor status, scan data, and robot state updates. UART/WiFi is one of the required mapped components.
Other Platform Components or Modes (may be novel features for bonus points) Timers will coordinate scan sweeps, periodic sensor updates, and movement timing. Servo control may be used to rotate the scan sensor for field sweeps. Additional logic will manage command execution, emergency stop handling, and scan/drive modes	

Elements of the Test Field

Briefly describe a test field in the context of the real application (e.g., Martian terrain, city streets, etc.). Then state what each of the basic objects and other elements required for the test field represent in terms of the AV application. Draw and attach a **sketch of a possible simple test field for the lab**.



Scanned with CamScanner

Test Field Description
The test field represents a hazardous remote-operation environment in which the operator cannot directly view the rover's path and must instead rely on transmitted sensor data, scan results, and GUI feedback. The rover starts in an unknown position and must be driven through obstacles and hazards to a final goal zone, where it must be positioned accurately. This fits the project requirement that teams cannot see the obstacle field during the demonstration and must rely on transmitted information to navigate.

Basic Objects and Other Elements	Mapping to AV Application Test Field

Tall objects (wide or composite)	These represent major obstacles such as walls, large rocks, or equipment structures that the rover must detect and avoid completely. The project requires the robot not to contact these.
Short objects	These represent low-profile debris or small floor obstacles that may not always be visible to long-range sensing. Slight contact may occur, but the rover should recover and navigate around them if detected.
Holes	These represent dangerous drop-offs, pits, or unstable terrain zones. The rover must detect and avoid these using cliff-related sensing so it does not fall in.
Pillars (thin tall object)	These represent the markers of the destination zone. The rover must use them to identify the goal area and then position itself inside without significantly disturbing them.
Out of bounds	These represent unsafe or invalid operating areas that the rover must not cross. In the real application context, they can be treated as no-go terrain, restricted zones, or edges of the allowed mission area. Boundaries are marked with white tape in the project.
Destination zone	This represents the final command objective where the rover must stop and signal that it has arrived and is properly positioned. For this application, it can be described as the final command post, scan checkpoint, or extraction zone.
Other Application Specific Elements (may be novel features for bonus points or incidents to avoid) Approximate map overlays in the GUI represent the operator's growing understanding of the field. Scan sectors and detected obstacle markers represent interpreted sensor results rather than direct camera vision. Speed-control and steering-tuning modes help the operator handle both broad navigation and fine positioning.	

Novel Features (Optional)

Novel Features	**Bonus Points**
Python graphical command center instead of a basic text terminal. Live approximate field mapping from scan data. Keyboard-driven remote piloting with adjustable speed and sensitivity. Integrated display of scan results, movement state, and sensor state in one operator panel. These are consistent with the project's bonus categories for GUI and additional functionality.	GUI: 5

Sketches

Attach the following sketches to your submission. These were noted above in red.

- **Problem sketch**
- **Technical system sketch**
- **Test field sketch**

Visual communication is helpful for sharing information. There are many ways to represent your information. The problem sketch could be anything from a cartoon (informal) to a UML use case diagram (more formal). The technical system sketch is similar to what has been used/shown in class and lab. The test field sketch probably needs no explanation.